Software Re-Engineering

Fall 2026

Lecture 6

Software & Data Restructuring

Software Restructuring

- Software restructuring is the process of modifying the internal structure of existing software without changing its external behavior or functionality.
- The main goal is to improve the readability, maintainability, and understandability of the software, making future changes easier and less error-prone.
- Software restructuring improves how the code is organized and written, while keeping what the software does essentially the same.

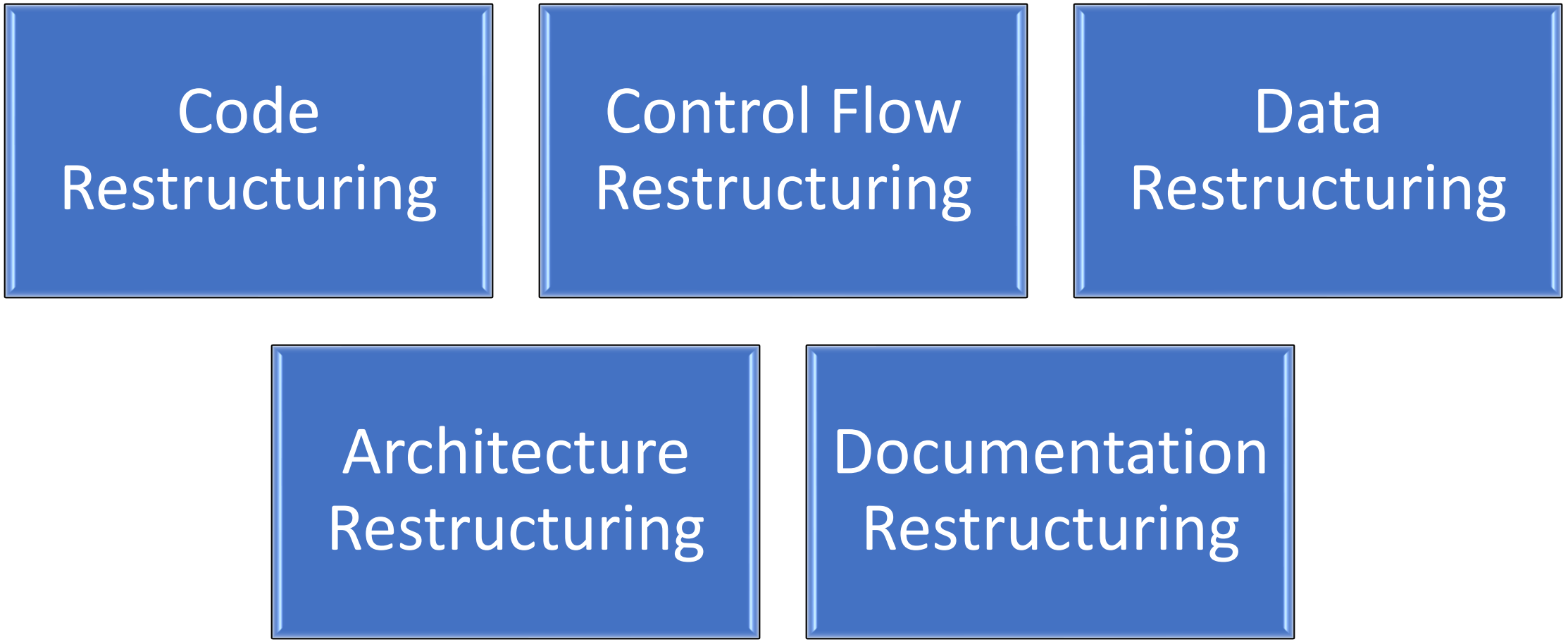
Software Restructuring

- Breaking large methods into smaller functions
- Renaming variables and methods for better clarity
- Removing duplicate or unnecessary code
- Simplifying complex logic
- Removing or reducing code smells
- Improving the organization of classes and modules

Importance of Software Restructuring

- As software systems grow, their code can become complex, difficult to understand, and harder to maintain.
- Software restructuring helps manage this complexity by improving the internal structure of the software without changing its functionality.
- It helps to:
  - Improve code readability and understandability
  - Reduce complexity and maintenance effort
  - Make future modifications easier and safer
  - Reduce the likelihood of errors
  - Extend the useful life of existing software systems

Types of Software Restructuring



Code Restructuring

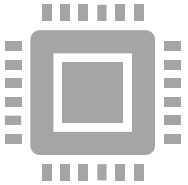
- Code restructuring focuses on improving the internal organization, readability, and quality of source code without changing its behavior.
- Common techniques:
  - Splitting large functions into smaller, cohesive functions
  - Renaming variables, methods, and classes
  - Removing duplicate or dead code
  - Simplifying complex expressions
  - Replacing magic numbers with named constants
  - Removing or reducing code smells
  - Improving cohesion and reducing unnecessary coupling
    - Example
      - A 300-line function performing several tasks can be divided into smaller functions such as *validateInput(), calculateTotal(),* and *generateReport().*

Code Restructuring



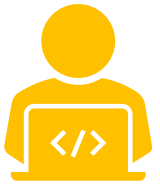
Refactoring

Refactoring is the process of improving the internal structure of code without changing its external behavior. The goal is to improve code readability, reduce complexity, and make the code easier to maintain.



Code Formatting

Improving the readability of code by adhering to standard formatting conventions is another aspect of restructuring. This can be automated by using tools that enforce code style guidelines.



Dead Code Elimination

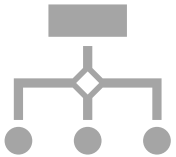
Dead code refers to sections of the code that are no longer in use, such as outdated methods, unreachable code, or variables that are declared but never used.

Code Restructuring



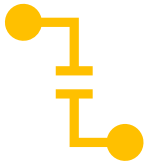
Code Simplification

Simplification refers to rewriting complex code structures in a more straightforward, readable way. This could involve reducing deep nesting, eliminating unnecessary conditionals, or replacing complicated algorithms with simpler, more efficient alternatives.



Code Modularization

Breaking down large, monolithic methods or classes into smaller, independent modules or functions is an important aspect of restructuring. This makes the code more modular, reusable, and easier to maintain.



Removal of redundant code

Code duplication often leads to difficulties in maintaining a system, as changes in one place must be replicated elsewhere. Restructuring eliminates duplicated code by introducing reusable methods or components.

Control Flow Restructuring

- Control flow restructuring focuses on improving the sequence and organization of program execution. The objective is to make program logic easier to understand, test, and maintain.
- Common techniques:
  - Simplifying deeply nested if-else structures
  - Replacing unnecessary goto statements
  - Simplifying complex conditional expressions
  - Converting complicated control structures into structured programming constructs
  - Reducing unnecessary loops and branches
  - Using guard clauses to handle exceptional conditions early

Control Flow Restructuring

Without a Guard Clause

```
void calculateAverage(int marks, int subjects)
{
    if (subjects > 0)
    {
        float average = marks / subjects;
        cout << "Average = " << average;
    }
}
```

With a Guard Clause

```
void calculateAverage(int marks, int subjects)
{
    if (subjects <= 0)
        return;

    float average = marks / subjects;
    cout << "Average = " << average;
}
```

Data Restructuring

- Data restructuring focuses on improving the organization, representation, and management of data used by the software.
- It can involve both program data structures and persistent data such as databases.
- Common techniques:
  - Replacing inappropriate data structures
  - Normalizing database tables
  - Splitting large data structures into logical components
  - Removing redundant data
  - Improving relationships between database entities
  - Changing data representation to improve maintainability
  - Encapsulating data using appropriate classes or data types

Data Restructuring

- string name;
- string address;
- string phone;

- class Customer {
- string name;
- string address;
- string phone;
- };

Elimination of goto Statements

- In programming languages, goto statements are used to jump from one part of the program to another.
- **Problem**

  Hard to follow: goto statements can make it hard to trace the program's flow since they jump from one part of the code to another, often breaking the logical sequence.

  Error-prone: It's easy to introduce bugs with uncontrolled jumps between sections of code.
- **Solution**
  - Replace goto statements with structured control flow constructs like if-else statements, loops, or functions.

Reducing Nested Conditionals

- Deeply nested if-else structures or loops make code harder to read and maintain. The more levels of nesting, the more difficult it is to track the logic and understand what conditions must be met for a particular block of code to execute.
- **Problem**
  - **Hard to read:** Code with multiple levels of nesting quickly becomes hard to follow.
  - **Difficult to debug:** Errors buried deep in nested structures are harder to trace.
- **Solution**
  - Use guard clauses or early returns to reduce nesting by handling special cases early and exiting the method or loop, allowing the main logic to proceed in a straightforward manner.

Loop Simplification

- Loops are a fundamental control flow structure, but overly complex loops such as nested loops or loops with complex exit conditions can reduce code clarity and performance. Loop simplification involves restructuring loops to make them easier to read and more efficient.
- **Problem**
  - Nested loops can quickly become difficult to manage, especially when they contain many iterations and condition checks.
  - Complex exit conditions can lead to hard-to-track errors or infinite loops.
- **Solution**
  - Break complex loops into smaller, more manageable structures or reduce nesting by separating logic into separate functions.

Replacing Switch/Case with Polymorphism or Functions

- Long switch-case or if-else chains can often be replaced with more structured control flow techniques, such as using polymorphism or strategy patterns in object-oriented programming.
- **Problem**
  - **Difficult to scale:** As the number of cases increases, the switch block becomes harder to manage.
  - **Violation of Open-Closed Principle:** Adding new cases requires modifying existing code, which is not scalable.
- **Solution**
  - Use polymorphism or separate the logic into smaller functions, each handling a specific case or action.

Breaking Long Methods into Smaller Functions

- Methods that try to do too much often have complex control flows that are difficult to understand. Breaking down long methods into smaller, single-responsibility functions improves readability and control flow.
- **Problem**
  - **Hard to test:** Long methods are difficult to test because they have too many responsibilities.
  - **Difficult to maintain:** Adding new functionality to long methods often results in more complexity, making the method even harder to manage.
- **Solution**
  - Extract smaller methods that handle a specific part of the logic, each focusing on one task.

Architecture Restructuring

- Architecture restructuring operates at the system or component level rather than individual lines of code. It improves how major components, modules, and services are organized and interact.
- Common techniques:
  - Decomposing large modules into smaller modules
  - Reducing dependencies between components
  - Improving cohesion and reducing coupling
  - Introducing appropriate architectural layers
  - Reorganizing components and interfaces
  - Migrating from tightly coupled designs to more modular architectures
  - Restructuring a monolithic system into services or components where appropriate

Architecture Restructuring

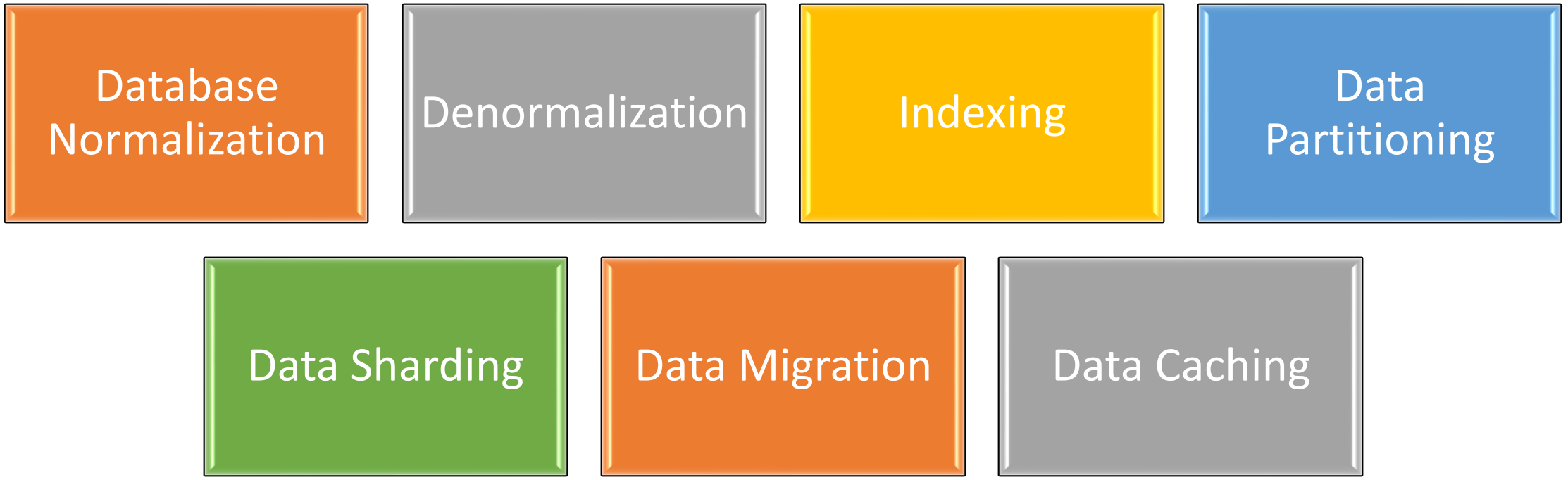
- User Management
- Order Processing
- Payment
- Inventory
- Reporting

- User Service
- Order Service
- Payment Service
- Inventory Service
- Reporting Service

Documentation Restructuring

- Documentation restructuring focuses on improving the organization, accuracy, and usefulness of software documentation.
- This is particularly important in legacy systems where documentation may no longer match the actual implementation.
- Common techniques:
  - Updating outdated technical documentation
  - Removing obsolete documentation
  - Improving API documentation
  - Updating architecture diagrams
  - Documenting complex algorithms and dependencies
  - Improving code comments where necessary
  - Generating documentation from source code where possible

Data Restructuring



Database Normalization

- It is the process of structuring a relational database to reduce redundancy and improve data integrity. It involves organizing the data into tables in a way that minimizes duplicate data, ensures data dependencies are logical, and makes the database easier to maintain.
- Normalization is typically achieved by applying a series of rules, called normal forms, to divide data into smaller, related tables. Each normal form addresses specific issues related to redundancy, dependency, and efficiency.
  - First Normal Form (1NF)
  - Second Normal Form (2NF)
  - Third Normal Form (3NF)
  - BCNF (Boyce–Codd Normal Form)
  - Fourth Normal Form (4NF)
  - Fifth Normal Form (5NF)

First Normal Form (1NF)

- Ensures that each table contains only atomic or indivisible values and that there are no repeating groups or arrays.
  - A table storing customer names and their orders, instead of using multiple columns for orders, would separate orders into another table linked by a foreign key.

Customer_ID	Customer_Name	Phone_Numbers
1	Ali	123456789, 24687531
2	Ayesha	987654321

Customer_ID	Customer_Name	Phone_Numbers
1	Ali	123456789
1	Ali	24687531
2	Ayesha	987654321

Second Normal Form (2NF)

- Ensures that the table is in 1NF and that all non-primary key attributes are fully functionally dependent on the primary key.
  - Splitting a customer table that contains both customer information and their order details into two tables
    - Customers
    - Orders
  - linked by a foreign key, to avoid repetition of customer details.

Second Normal Form (2NF)

Customer_ID	Customer_Name	Order_ID	Order_Date	Product
1	Ali	1001	04-10-2024	iPhone
1	Ali	1002	05-10-2024	Tablet
2	Ayesh	1003	06-10-2024	Power Bank

Customer_ID	Customer_Name
1	Ali
2	Ayesha

Order_ID	Customer_ID	Order_Date	Product
1001	1	04-10-2024	iPhone
1002	1	05-10-2024	Tablet
1003	2	06-10-2024	Power Bank

Third Normal Form (3NF)

- Ensures that the table is in 2NF and that all the attributes are not only dependent on the primary key but also independent of each other (removal of transitive dependency).
  - Moving an address field from an employee table to a separate Address table, where addresses are not dependent on other non-key fields like employee name

Third Normal Form (3NF)

Employee_ID*PK	Employee_Name	Department_ID	Department_name	Office_Location
1	Ali	RE001	HR [Department_ID]	Block A [Department_ID]
2	Hamza	RE002	IT	Block C
3	Ayesha	RE003	HR	Block A

Employee_ID	Employee_Name	Department_ID
1	Ali	RE001
2	Hamza	RE002
3	Ayesha	RE003

Department_ID*FK	Department_name	Office_Location
RE001	HR	Block A
RE002	IT	Block C
RE003	HR	Block A

Fourth Normal Form (4NF)

- 4NF focuses on eliminating multi-valued dependencies, which occur when a column has multiple independent sets of values for the same record.
- In 4NF, a table should not contain more than one independent multi-valued fact about an entity.

Fourth Normal Form (4NF)

Student_ID	Student_Name	Department	Club
1	Ali	CS	Sports Club
1	Ali	CS	Music Club
2	Ayesha	SE	Arts Club
2	Ayesha	SE	Chess Club

Student_ID	Student_Name	Department
1	Ali	CS
2	Ayesha	SE

Student_ID	Student_Name	Club
1	Ali	Sports Club
1	Ali	Music Club
2	Ayesha	Arts Club
2	Ayesha	Chess Club

Fourth Normal Form (4NF)

- Ali has multiple clubs
- Ali has one department [CS], which is repeated for each club, causing redundancy.
- The Club and department are independent facts about Ali, but they are stored in the same table.
- This leads to unnecessary duplication of data and violates 4NF because the student has two independent multi-valued facts: their departments and their clubs.
  - Imagine Ali joins a new club, like the Robotics Club. In the original design, you would need to create a new row with Alice's department again.
  - In the 4NF design, you would only add one row in the Student_Club table
  - There's no need to repeat his department, making it more efficient.

Fifth Normal Form (5NF)

- It deals with cases where relationships between data can be reconstructed from smaller tables without creating redundancy.
- In 5NF, a table should be divided into smaller related tables without losing data or introducing redundancy, ensuring that the data can be reconstructed by joining those smaller tables.

Fifth Normal Form (5NF)

Student_ID	Course	Faculty
1	SE	Dr. Asad
1	SRE	Dr. Zulfiqar
2	DSA	Dr. Asad
2	SPM	Dr. Zohaib

Student_ID	Course
1	SE
1	SRE
2	DSA
2	SPM

Student_ID	Faculty
1	Dr. Asad
1	Dr. Zulfiqar
2	Dr. Asad
2	Dr. Zohaib

Denormalization

- It is the process of introducing redundancy into a database to improve performance, particularly in read-heavy systems. It is often used in systems where joining tables is costly in terms of performance.
  - **Merging Tables:** Combine related tables into a single table to avoid the overhead of joins.
    - In a reporting system, instead of joining Orders and Customers tables, denormalize the data into a single CustomerOrders table that stores both customer and order information.
  - **Adding Redundant Columns:** Add calculated or derived columns to store frequently accessed values.
    - Storing the total order value in the Orders table, even though it could be calculated by summing the values of individual order items.
  - **Precomputed Aggregates:**
    - Store the results of aggregate functions like sums, counts, or averages in the database to avoid recalculating them each time a query runs.
      - A ProductSales table might store total sales for each product to avoid summing up individual sales every time a report is generated.

Indexing

- It is a technique used to improve the speed of data retrieval operations in a database by creating an index (data structure) on columns frequently used in queries.
  - **Single-column Indexes:** Indexes created on a single column to improve the performance of queries that filter or sort by that column.
    - Indexing the customer_id column in an Orders table to speed up queries that retrieve orders for a specific customer.
  - **Composite Indexes:** Indexes created on multiple columns to optimize queries that filter by multiple criteria.
    - Creating a composite index on the product_id and order_date columns in an OrderDetails table to improve performance for queries that filter by both product and date.
  - **Full-Text Indexes:** Specialized indexes that allow for fast searches on large text fields.
    - In a blog application, a full-text index can be applied to the content column of posts to enable fast text searches.

Data Partitioning

- It is the practice of dividing a large database table into smaller, more manageable pieces (partitions) for performance and scalability improvements. This is particularly useful in systems that handle large amounts of data.
  - **Range Partitioning:** Dividing data into partitions based on a range of values in a particular column (e.g., date or ID range).
    - Partitioning an Orders table based on the order date, with one partition per year.
  - **Hash Partitioning:** Partitioning data based on a hash function applied to a column's values.
    - Partitioning a Users table based on a hash of the user_id to evenly distribute the data across partitions.
  - **List Partitioning:** Partitioning data based on a predefined list of values.
    - Partitioning a Products table based on product category, with each partition storing data for a specific category (e.g., electronics, clothing).

Data Sharding

- It is a horizontal partitioning technique where a database is divided into smaller databases (shards), each running on a different server. Sharding is essential for scaling databases when a single server can no longer handle the data load.
  - **Range-based Sharding:** Data is distributed across shards based on a range of values.
    - Sharding a Customers table by splitting customers based on their last names (e.g., A-M in one shard and N-Z in another).
  - **Hash-based Sharding:** Data is distributed across shards based on a hash function applied to a column, typically the primary key.
    - Hashing the user_id to distribute users across multiple shards.
  - **Geographical Sharding:** Data is partitioned based on geographical location, where each shard serves data specific to a region.
    - A global application might shard its user data by country, with each country's data stored in a separate shard.

Data Migration

- It is the process of moving data from one system to another, often involving restructuring the data to fit into a new format, schema, or database system.
  - **ETL (Extract, Transform, Load):** This technique involves extracting data from the source, transforming it to fit the new schema or system, and loading it into the target database.
    - Migrating data from a legacy database (e.g., Oracle) to a modern database (e.g., PostgreSQL) by first extracting, cleaning, and transforming the data.
  - **Incremental Data Migration:** Migrating data in batches or chunks, often while the system remains in use. This ensures minimal disruption during the migration process.
    - Migrating customer records in batches, moving inactive customers first, followed by active customers.
  - **Database Replication:** Copying data from one database to another in real-time or near real-time, which can serve as a preface to migration.
    - Replicating a live database to a new system before fully cutting over to the new database.

Data Caching

- It is a technique where frequently accessed data is temporarily stored in memory to reduce the time needed to retrieve it from a database. Data caching is particularly useful for improving the performance of read-heavy applications.
  - **In-memory Caching:** Storing frequently accessed data in memory (RAM) for faster access.
    - Using Redis or Memcached to store user session data, reducing the number of database reads.
  - **Database Query Caching:** Storing the results of a database query to avoid running the same query multiple times.
    - Caching the result of a complex report query to speed up subsequent requests.
  - **Page-level Caching:** Storing entire pages of data, such as HTML pages, so that they can be served without querying the database.
    - Caching the homepage of a news website to reduce database load during peak traffic.

Questions?